



浙江大學
ZHEJIANG UNIVERSITY

Google Chrome Dino Game Based on FPGA

数字逻辑设计课程设计项目报告

2026 年 7 月 3 日

2025-2026 春夏学期

[Visit Our Repository on GitHub](#)

目录

I.	背景介绍	3
II.	设计说明	3
II.1.	设计开发环境	3
II.2.	输入输出交互	3
II.3.	总体设计思路	3
II.4.	成员分工与贡献比例	4
III.	模块结构说明	5
III.1.	顶层模块 top	5
III.2.	PS/2 键盘输入模块	5
III.3.	游戏主状态机模块	5
III.4.	恐龙运动与姿态控制模块	6
III.5.	物体运动系统	6
III.6.	随机生成与昼夜系统	6
III.7.	碰撞检测系统	7
III.8.	计分与音效系统	7
III.9.	VGA 控制与显示系统	7
IV.	仿真结果	8
IV.1.	PS/2 键盘输入模块仿真	8
IV.2.	主状态机模块仿真	8
IV.3.	物体运动系统仿真	9
V.	上板验证结果	10
V.1.	PS/2 键盘输入模块上板验证	10
V.2.	主状态机模块上板验证	10
V.3.	物体运动系统上板验证	11
V.4.	综合上板验证	13
VI.	开发与调试过程分析	16
VI.1.	坐标与素材尺寸	16
VI.2.	PS/2 输入	16
VI.3.	状态机	16
VI.4.	VGA 显示	16
VI.5.	生成请求跨时钟周期	16
VII.	程序流程与操作说明	17
VII.1.	程序流程	17
VII.2.	游戏操作说明	17
VIII.	总结	17

I. 背景介绍

本次课程设计选择实现 Google Chrome 离线小恐龙游戏。该游戏同时包含键盘输入、状态机控制、随机事件生成、运动更新、碰撞检测、计分系统、VGA 图像显示和音效输出，能够较完整地体现时序逻辑系统的设计过程。

游戏的基本规则是：玩家通过键盘控制恐龙跳跃或下蹲，躲避从右向左运动的仙人掌和翼龙；游戏开始后分数持续增加，随着分数提高，障碍物运动速度逐渐加快；当恐龙与障碍物发生碰撞时，游戏进入结束状态，并在屏幕上显示 Game Over 与重开提示。

为了使画面更接近原版游戏，本项目还加入了地面滚动、恐龙跑步动画、翼龙扇动动画、云朵背景、昼夜切换、月相变化和蜂鸣器音效。

II. 设计说明

II.1. 设计开发环境

- 实验平台：Sword Kintex7 (Part: xc7k160tffg676-1)
- 开发环境：Xilinx Vivado
- 硬件描述语言：Verilog HDL
- 显示分辨率：VGA 640×480
- 系统时钟：板载 100MHz 时钟
- VGA 像素时钟：由 100MHz 四分频得到约 25MHz

II.2. 输入输出交互

类型	信号/设备	功能
输入	PS/2 键盘	空格/上方向键控制跳跃，下方向键控制下蹲，P 键控制暂停
输入	RSTN	全局复位，低电平有效
输出	VGA	输出 640×480 像素游戏画面
输出	Buzzer	输出跳跃、碰撞、得分提示音效

Table 1 — 输入输出交互信号

在核心模块验证阶段，为了便于观察内部状态，还单独设计了若干上板验证顶层，使用 Sword 平台上的 SW、BTN、LED 和四位七段数码管显示状态机、PS/2 接收和运动模块的中间结果。

II.3. 总体设计思路

系统总体分为两层：

第一层是顶层连接模块 `top.v`。该模块本身不直接修改游戏状态，负责连接游戏逻辑系统与显示系统。

第二层包括游戏逻辑层 `dino_game_logic.v`。该层负责 PS/2 键盘解析、游戏主状态机、随机生成、物体运动、恐龙跳跃、碰撞检测、计分和音效触发。游戏逻辑的输出是当前游戏状态、恐龙位置、障碍物位置、背景元素位置、分数和音效信号。

第二层还包括显示与音效层 `display_system.v`。该层负责 VGA 时序控制、精灵 ROM 读取、像素区域判断、透明像素处理、像素优先级合成、分数显示、Game Over 界面和蜂鸣器输出。显示层不改变游戏状态，只根据逻辑层输出的信息生成当前扫描像素的 RGB 值。

这种“逻辑计算”和“像素渲染”分离的结构，使游戏逻辑可以以帧为单位更新，而 VGA 显示可以按像素扫描连续输出，两者通过稳定的接口进行通信。

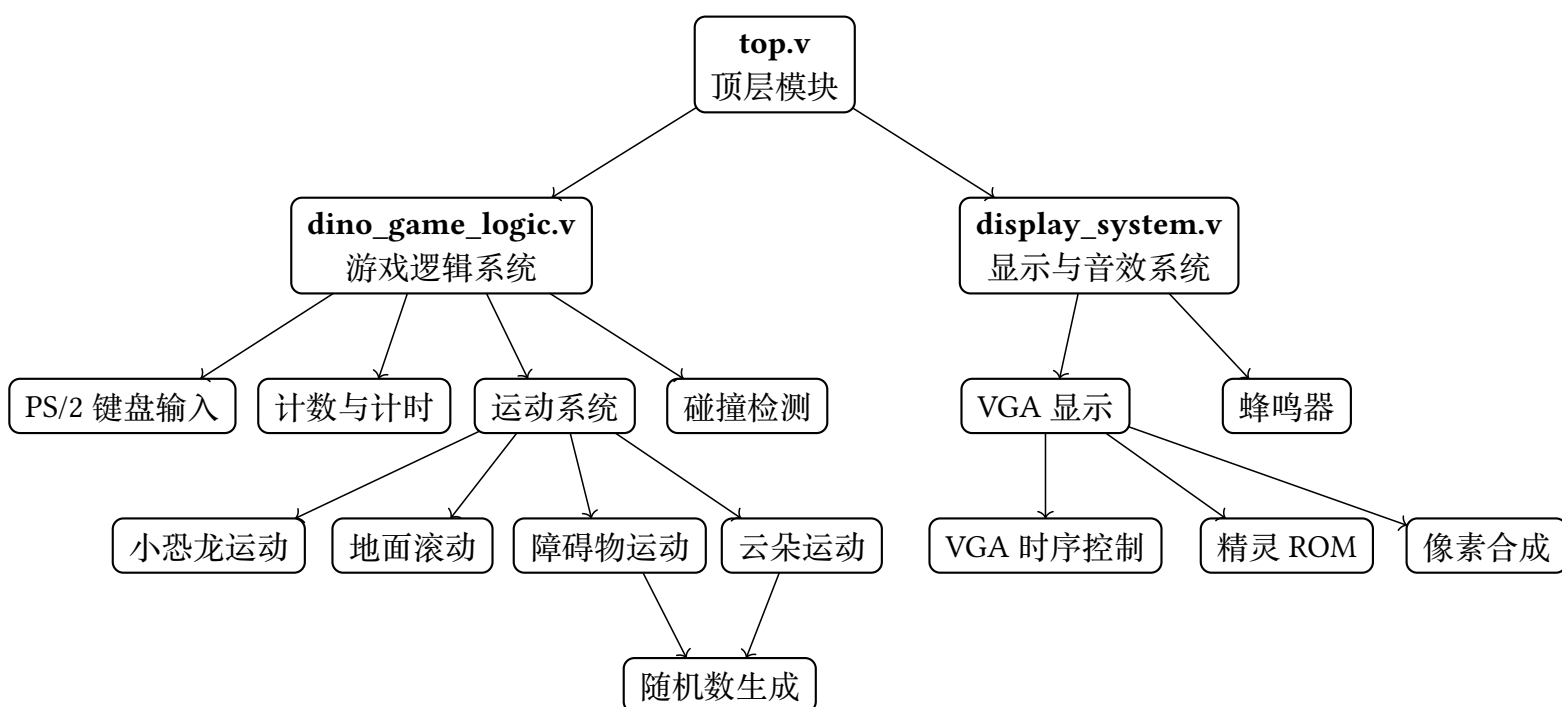


Figure 1 — 项目模块层次结构示意图

II.4. 成员分工与贡献比例

- 成员 A (50%): 主要负责游戏逻辑层设计，包括 PS/2 键盘输入、主状态机、恐龙跳跃、随机生成、物体运动、碰撞检测、计分系统和音效触发；同时负责编写游戏逻辑相关仿真文件，并参与上板调试。
- 成员 B (50%): 主要负责显示系统设计，包括 VGA 控制器、精灵 ROM、像素区域判断、透明像素处理、像素合成、昼夜/月相显示、分数显示和 Game Over 界面；同时负责图片素材处理和 `.mem` 文件生成。

二人共同负责顶层集成、约束文件整理、上板验证、调试、报告整理和演示视频制作。

III. 模块结构说明

III.1. 顶层模块 top

`top.v` 是系统的最高层模块，输入包括 100MHz 时钟、复位信号和 PS/2 键盘信号，输出包括 VGA 三色信号、行同步、场同步以及蜂鸣器信号。顶层模块实例化 `dino_game_logic` 和 `display_system`。

其中 `display_system` 内部的 VGA 控制器每一帧结束时输出单周期 `frame_end` 脉冲，该信号反馈给 `dino_game_logic`，作为游戏状态统一更新的节拍。这样可以保证物体坐标、分数和随机生成均在帧边界更新，避免在一帧显示过程中出现同一元素坐标跳变的问题。

III.2. PS/2 键盘输入模块

键盘输入由 `dino_key_input` 完成，内部包含 `ps2_byte_rx` 和 `ps2_dino_key` 两部分。

`ps2_byte_rx` 负责 PS/2 协议的底层接收。PS/2 每一帧由起始位、8 位数据位、奇校验位和停止位构成，接收模块在 PS/2 时钟下降沿采样数据。模块对 PS/2 时钟进行同步处理，并在接收完整一帧后检查停止位和奇校验，若校验正确则输出扫描码 `code` 和有效信号 `code_vld`，否则输出错误标志 `code_err`。

`ps2_dino_key` 负责将扫描码转换为游戏按键信号。本项目识别扫描码集合 2 中的空格键、上方向键、下方向键和 P 键。其中空格键用于开始游戏和跳跃，上方向键也可触发跳跃，下方向键用于下蹲，P 键用于暂停。模块区分通码和断码，能够输出按键保持状态 `*_key`，也能在按键刚被按下时输出单周期触发信号 `*_trig`。长按产生的重复通码不会重复产生触发脉冲，从而避免一次长按造成多次跳跃或状态跳转。

III.3. 游戏主状态机模块

游戏主状态机由 `game_fsm` 实现，包含四个状态编码：

状态	编码	含义
S_IDLE	00	待机状态
S_PLAY	01	游戏进行状态
S_PAUS	10	暂停状态
S_OVER	11	游戏结束状态

Table 2 — 主状态机

复位后状态机进入待机状态。待机状态下，按下空格或上方向键后，在下次 `frame_end` 到来时进入游戏进行状态。游戏进行状态下，如果检测到碰撞信号 `hit_flag`，则在帧边界进入游戏结束状态；如果检测到 P 键触发，则进入暂停状态。游戏结束状态下，再次按下空格或上方向键即可重新开始。

由于键盘触发信号只持续一个系统时钟周期，而状态机只在 `frame_end` 时刻更新，如果直接使用 `space_trig`，就可能漏掉两帧之间的按键。因此状态机中增加了 `start_req` 锁存逻辑。该锁存只在待机和游戏结束状态下记录开始请求，在游戏进行状态下会被清除。

III.4. 恐龙运动与姿态控制模块

恐龙的横坐标在显示系统中固定，纵坐标由 `dino_game_logic` 中的跳跃逻辑维护。项目使用简单的离散物理模型描述恐龙跳跃过程：起跳时给予向上的初速度 `JUMP_V0`，之后每一帧速度受到重力 `GRAVITY` 影响逐渐改变，直到恐龙回到站立高度。若恐龙在空中时按住下方向键，则额外加入 `FAST_FALL`，实现快速下落效果。

恐龙在地面上时，如果按下下方向键，则进入下蹲状态；跳跃过程中不会重复响应新的跳跃触发，防止空中二段跳。恐龙的显示姿态由 `dino_state` 表示，包括静止/跳跃姿态、跑步左脚、跑步右脚、下蹲左脚、下蹲右脚和死亡姿态。跑步和下蹲动画通过一个帧计数器控制，每隔若干帧翻转一次动画位。

III.5. 物体运动系统

物体运动系统由 `dino_obj_motion` 封装，内部包括地面、仙人掌、翼龙和云朵四类运动模块。

地面运动模块 `ground_motion` 维护两段宽度为 900 像素的地面贴图。两段地面初始坐标分别为 0 和 900，每帧按当前速度向左移动。当某一段完全移出屏幕左侧后，将其接到另一段右侧，从而形成连续滚动地面。

仙人掌运动模块 `cactus_motion` 维护 3 个仙人掌槽位。每个槽位包含有效位、横坐标和类型。收到 `cactus_new` 后，模块选择第一个空槽位，在屏幕右侧生成新仙人掌。仙人掌速度与地面速度相同，移出屏幕后清空槽位。仙人掌类型包括 3 种小仙人掌和 3 种大仙人掌，对应不同的 ROM 资源。

翼龙运动模块 `ptero_motion` 维护 2 个翼龙槽位。翼龙速度设为 `speed_px + 1`，略快于地面和仙人掌。模块还包含翅膀动画计数器，每 12 帧翻转一次翅膀状态，约为 0.2 秒切换一次动画帧。翼龙的纵坐标由随机系统给出，可产生高、低两种飞行高度。

云朵运动模块 `cloud_motion` 维护 3 个云朵槽位。云朵属于背景元素，速度约为 `speed_px / 2`，且最低为 1 像素/帧。云朵随机出现在屏幕上方不同高度，移出屏幕后清空槽位。

III.6. 随机生成与昼夜系统

随机系统由 `dino_rand_sys` 实现，内部使用 30 位 Fibonacci LFSR 产生伪随机数。随机系统只产生“生成请求”，不直接维护障碍物坐标。运动系统收到生成请求后，再选择空槽位并设置初始位置。

障碍物生成需要同时满足随机触发、最小间距和槽位可用三个条件。最小间距根据当前速度和恐龙最长滞空时间估算，避免出现理论上无法跳过的障碍组合。系统通过 `obs_gap` 记录距离上一个障碍物已经经过的像素距离，只有当间距足够时才允许生成新障碍。障碍类型在仙人掌和翼龙之间按序列变化，既保证随机性，也避免长时间只出现一种障碍物。

昼夜系统通过 `day_night_cycle` 记录当前昼夜阶段，每一定时间切换一次阶段；当昼夜周期完整推进后，月相 `moon_phase` 随之变化。显示系统根据 `night` 和 `moon_phase` 生成夜晚反色效果和月亮图像。

III.7. 碰撞检测系统

碰撞检测在 `dino_game_logic` 内部完成。系统只在游戏进行状态下检测碰撞。由于图片素材中包含透明留白，如果直接用整张图片矩形碰撞，会导致碰撞范围过大，影响游戏手感。因此本项目采用近似有效区域的多矩形碰撞方法。

恐龙在站立/跳跃状态下使用身体、头部和腿部等多个矩形近似有效区域；在下蹲状态下使用更宽、更矮的矩形近似。仙人掌按小仙人掌和大仙人掌分别设置主干、左枝和右枝的碰撞矩形；多株仙人掌由多个单株区域组合。翼龙则使用身体、头部和上下翅膀区域近似。最后通过通用 AABB 矩形重叠函数判断两组矩形是否相交。

这种方法比单一外接矩形更接近可见图像，既不会过早判定碰撞，也不会因为透明区域导致不合理的失败。

III.8. 计分与音效系统

分数系统使用二进制寄存器 `score_bin` 保存当前分数，并在显示前拆分为 5 位 BCD 数字 `score0` 到 `score4`。分数在游戏进行状态下按帧累加，且分数增长步长会随速度提高而增加。分数最大值为 99999，达到上限后饱和。

游戏速度由分数分段控制。初始速度为 7 像素/帧，随后在不同分数阶段提升到 8、9、10、11 像素/帧。地面、仙人掌、翼龙、云朵和随机间距判断均使用同一速度基准，保证游戏难度变化一致。

音效系统使用简单方波输出，包含跳跃音效、碰撞音效和阶段得分音效。每类音效使用独立的倒计时和分频计数器控制持续时间与频率。显示系统将三类音效信号或运算后驱动蜂鸣器。

III.9. VGA 控制与显示系统

VGA 控制由 `vga_ctrl` 实现。模块将 100MHz 系统时钟四分频为约 25MHz 像素时钟，并按照 640×480@60Hz 的标准 VGA 时序产生水平计数器、垂直计数器、行同步信号和场同步信号。在每帧最后一个扫描点处，模块输出单周期 `frame_end` 脉冲，作为游戏逻辑更新节拍。

显示系统采用精灵 ROM 和组合像素合成方式。各类图片素材预先转换为 `.mem` 文件，由 `sprite_rom` 模块通过 `$readmemh` 读入 FPGA 内部 ROM。每个像素通常用 2 bit 编码，其中 `00` 表示透明，其余编码表示不同灰度或颜色。显示系统根据当前扫描坐标判断像素是否落在某个精灵区域内，若命中则计算 ROM 地址并读出像素编码。

最终像素由 `compose_pixel` 函数按优先级合成。大体优先级为：背景色、地面、云朵、翼龙、仙人掌、恐龙、分数、暂停/结束界面。Game Over 字样和 Pause 字样使用矩形笔画组合生成，不额外占用 ROM。夜晚模式下，系统对已有像素整体反色，并在最后叠加月亮，使画面具有昼夜变化效果。

IV. 仿真结果

IV.1. PS/2 键盘输入模块仿真

PS/2 模块仿真文件为 `tb_dino_key_input.v`。仿真覆盖了空格键、上方向键、下方向键以及错误校验帧等情况。测试内容包括：

1. 发送空格键通码 29h 后，`space_key` 置 1，`space_trig` 只产生一次；
2. 长按空格导致再次发送 29h 时，不重复产生 `space_trig`；
3. 发送 F0 29 后，`space_key` 清零；
4. 发送扩展键 E0 75 后，上方向键状态置 1，触发脉冲产生一次；
5. 发送 E0 F0 75 后，上方向键释放；
6. 发送 E0 72 和 E0 F0 72 后，下方向键按下和释放均能正确识别；
7. 故意发送错误奇校验的空格帧，模块不会改变当前按键状态，也不会输出触发脉冲。

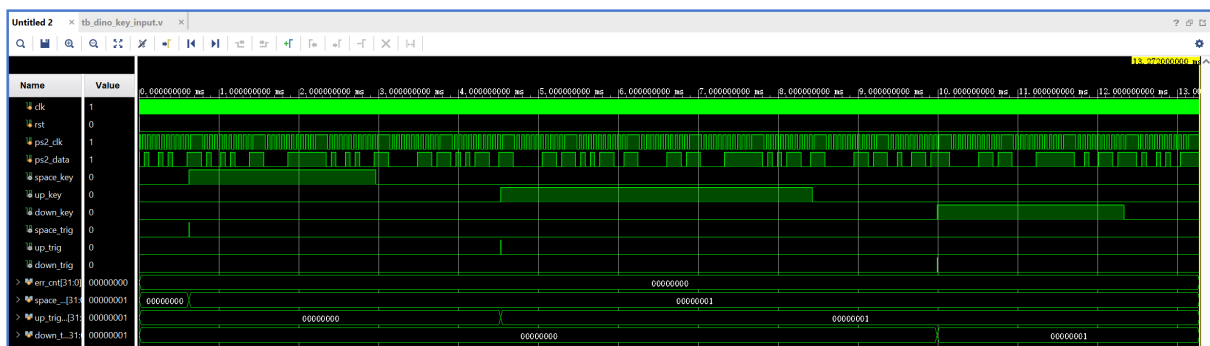


Figure 2 — ps/2 键盘输入模块仿真结果

IV.2. 主状态机模块仿真

主状态机仿真文件为 `tb_game_fsm.v`。仿真重点验证状态转换和请求锁存逻辑，尤其关注“游戏中按下空格后发生碰撞，是否会在游戏结束后立即重开”的问题。

测试过程包括：

1. 复位后进入 `S_IDLE`；
2. 在两帧间按下空格，状态不会立即改变，而是在下一次 `frame_end` 进入 `S_PLAY`；
3. 进入游戏时 `game_start` 只保持一个系统时钟周期；
4. 游戏进行状态下再次按空格不会改变状态，也不会残留为重开请求；
5. 游戏进行状态下 `hit_flag` 有效时，在帧边界进入 `S_OVER`；
6. 游戏结束状态下再次按空格后，在下一帧重新进入 `S_PLAY`；
7. `game_over` 也只保持一个系统时钟周期。

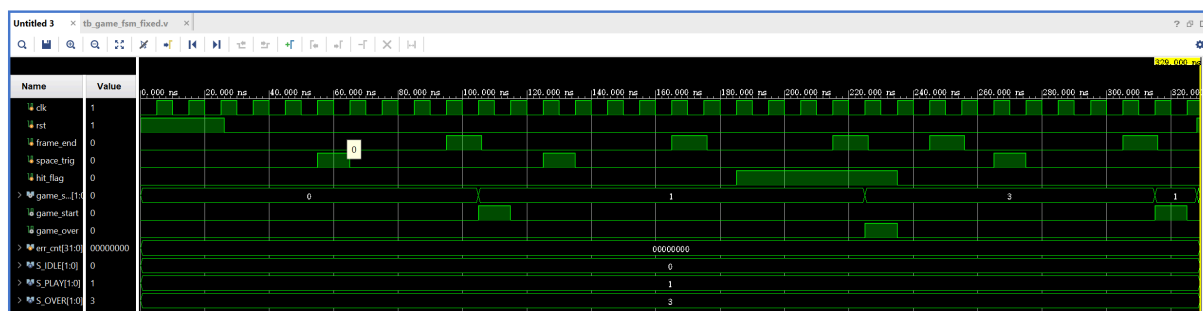


Figure 3 — 主状态机模块仿真结果

IV.3. 物体运动系统仿真

物体运动系统仿真文件为 `tb_dino_obj_motion.v`。仿真对象包括地面、仙人掌、翼龙和云朵的槽位更新与坐标变化。地面验证中，两段地面的初始坐标分别为 0 和 900，游戏开始后以 `speed_px` 为步长向左移动，当一段地面完全移出屏幕左侧后，会接到另一段右侧，形成连续滚动。

仙人掌验证中，收到 `cactus_new` 后，第一个空槽位变为有效，横坐标从 700 开始逐帧减小，类型保持为随机系统给出的类型；当仙人掌完全移出屏幕后，对应有效位清零。翼龙验证中，收到 `ptero_new` 后，翼龙进入空槽位，速度为 `speed_px + 1`，同时翅膀动画状态每 12 帧翻转一次。

云朵验证中，收到 `cloud_new` 后，云朵以约 `speed_px / 2` 的速度缓慢左移，体现远景背景元素运动较慢的效果。

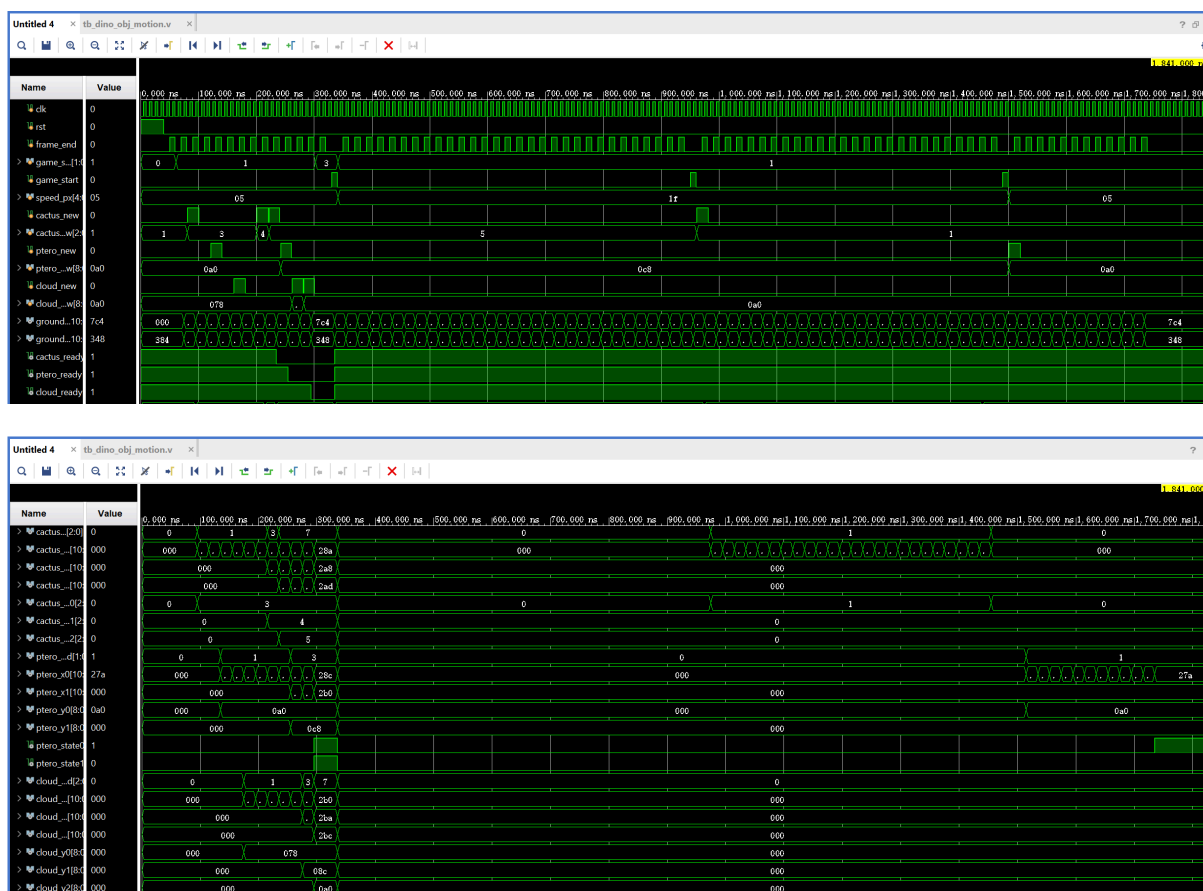


Figure 4 — 物体运动系统仿真结果

V. 上板验证结果

V.1. PS/2 键盘输入模块上板验证

PS/2 验证顶层中，LED 显示空格、上键、下键的保持状态和触发状态，七段数码管根据开关的选择显示当前按键状态、最近扫描码、有效帧计数等信息。通过该验证可以判断键盘是否正确发送扫描码，以及模块是否正确识别通码、断码和扩展码。

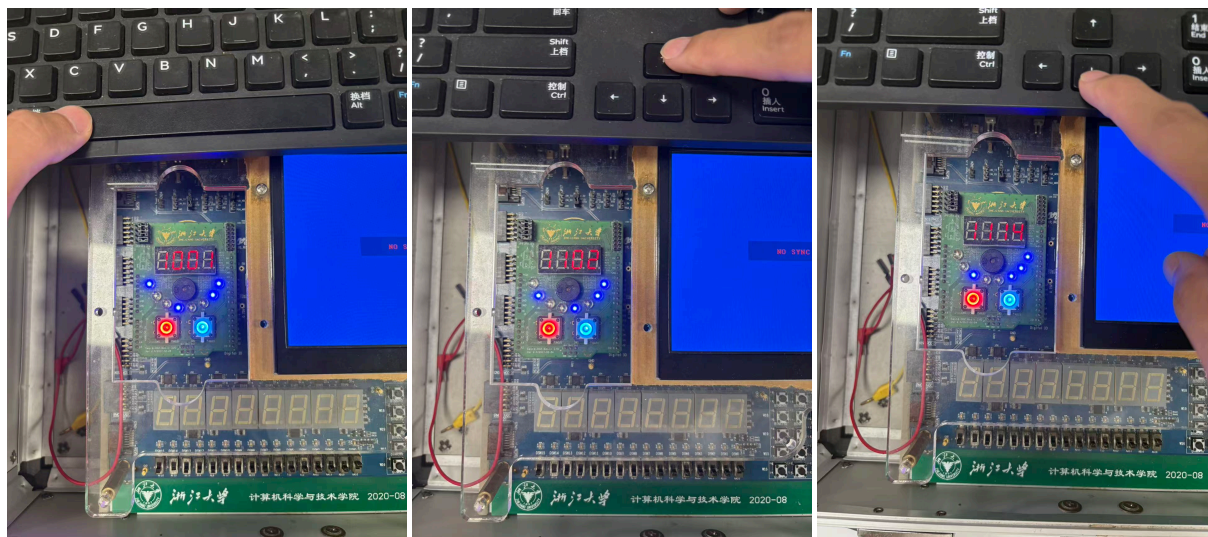


Figure 5 — ps/2 键盘输入模块上板验证结果

上图依次测试空格键、上键、下键。

上图中，LED[2:0] 表示对应按键的保持状态，LED[5:3] 表示对应拉长显示的按键触发状态，七段数码管最右位显示当前按键状态（1 为空格键，2 为上键，4 为下键）。

V.2. 主状态机模块上板验证

状态机验证顶层中，SW[15] 用作复位；BTN[0] 模拟开始/重开；BTN[1] 模拟碰撞（需长按）；BTN[2] 在手动模式下产生一帧 `frame_end`；SW[0] 选择手动帧或自动帧；SW[1] 选择低速自动帧（2Hz）或接近真实帧率（60Hz）。LED 显示当前状态、开始脉冲、结束脉冲、碰撞电平和帧脉冲，七段数码管显示状态和事件计数。



Figure 6 — 主状态机模块上板验证结果

上图从左到右、从上到下依次为：发出开始请求、游戏开始、发出碰撞信号、游戏结束、发出重新开始请求、游戏重新开始。

上图七段数码管最左位为帧数，最右位为当前状态（0 表示待机，1 表示游戏中，3 表示游戏结束）；LED[2] 表示帧结束信号，LED[3] 表示碰撞信号，LED[4] 表示游戏开始请求。

根据该上板验证结果，游戏主状态机能正确处理游戏信号，做出正确的状态转化。

V.3. 物体运动系统上板验证

运动系统验证顶层中，BTN 和 SW 用于手动触发生成事件、切换显示内容和调整验证模式，LED 与七段数码管显示槽位有效位、对象类型和坐标低位。通过逐帧观察，可以确认地面循环、障碍物左移、翼龙动画和云朵运动是否符合预期。

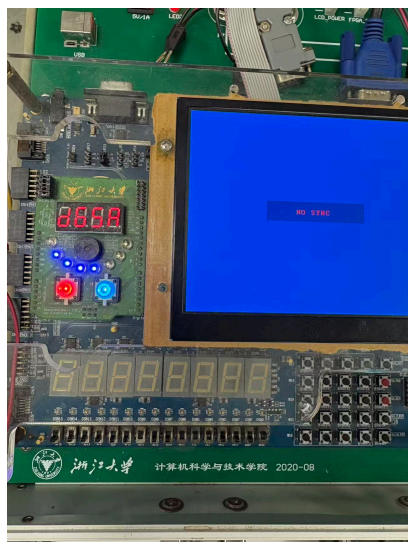


Figure 7 — 地面运动上板验证结果

上图中四位七段数码管显示地面的 X 坐标，能随着帧结束信号减小。

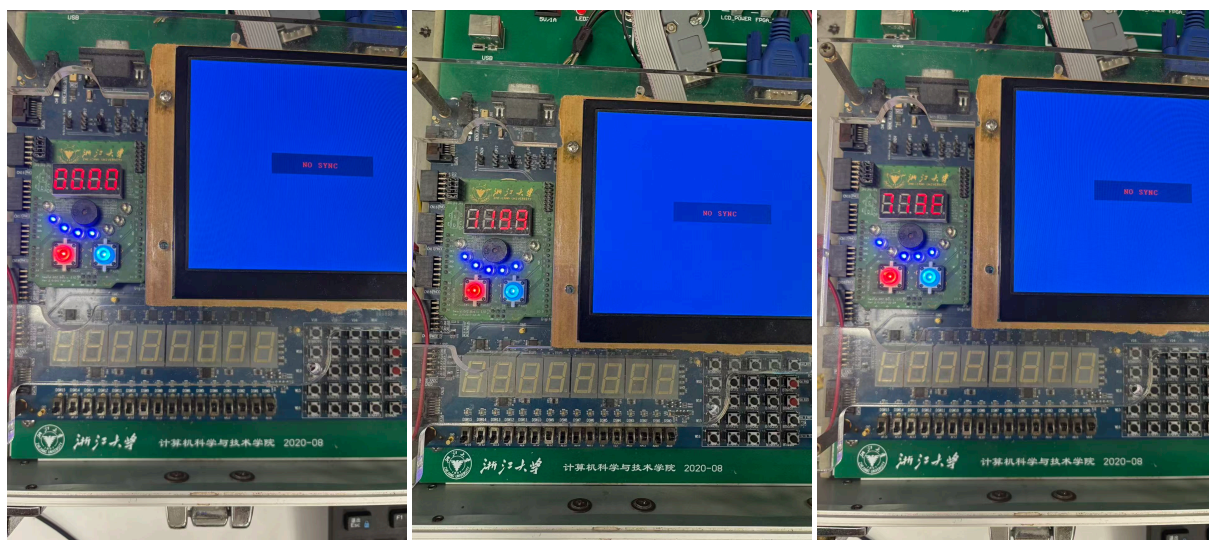


Figure 8 — 仙人掌运动上板验证结果

上图依次表示：未生成仙人掌、生成一个仙人掌、生成的仙人掌已满。上图中 LED[2] 表示仙人掌未满，LED[5] 表示至少生成了一个仙人掌。

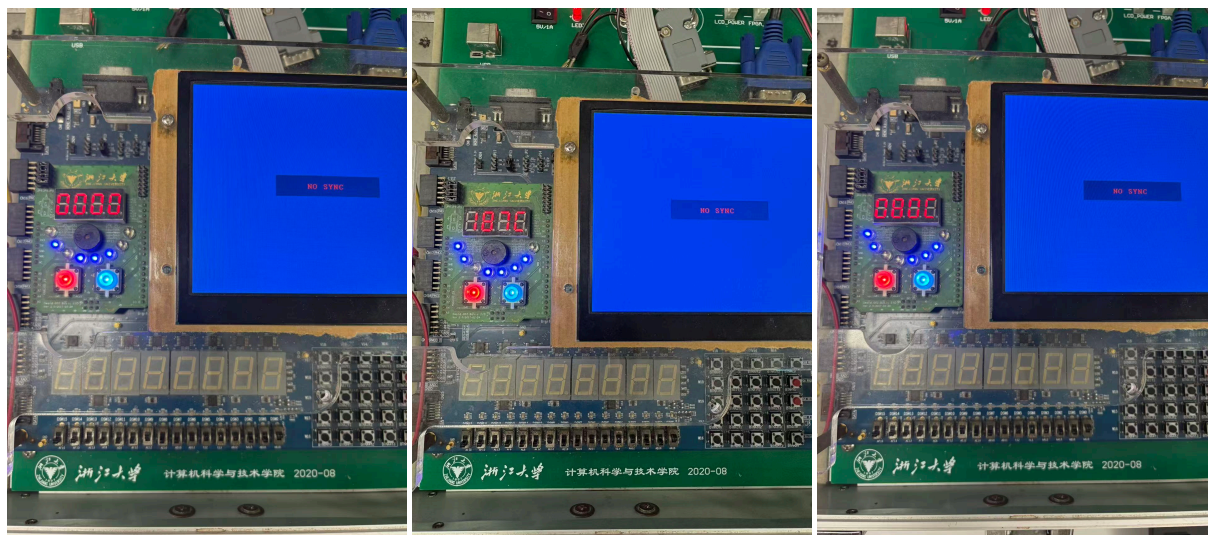


Figure 9 – 翼龙运动上板验证结果

上图依次表示：未生成翼龙、生成一个翼龙、生成的翼龙已满。上图中 LED[3] 表示翼龙未满，LED[6] 表示至少生成了一个翼龙。

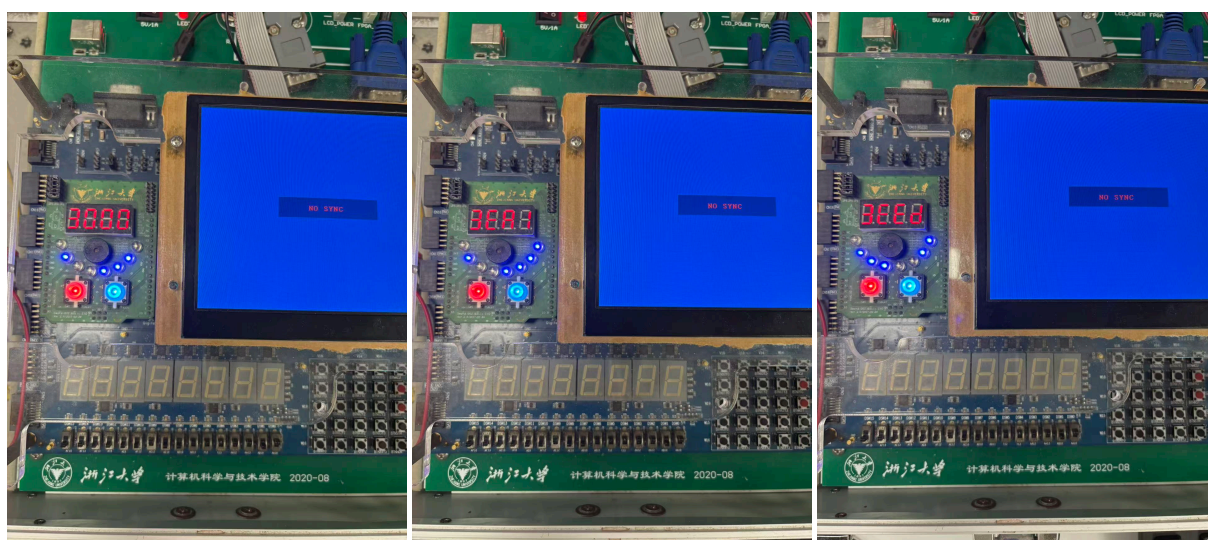


Figure 10 – 云朵运动上板验证结果

上图依次表示：未生成云朵、生成一个云朵、生成的云朵已满。上图中 LED[4] 表示翼龙未满，LED[7] 表示至少生成了一个翼龙。

物体运动系统上板验证结果表明，该系统可以正确给出各类物体是否存在、是否槽位已满，同时各物体的 X 坐标能随着帧结束信号减小。上板验证符合设计预期。

V.4. 综合上板验证

将验证符合预期的 Verilog 代码加入 Vivado 工程中，并将 .mem 文件制作为 IP 核，随后综合、实现、生成 Bitstream，导入实验板进行综合验证。

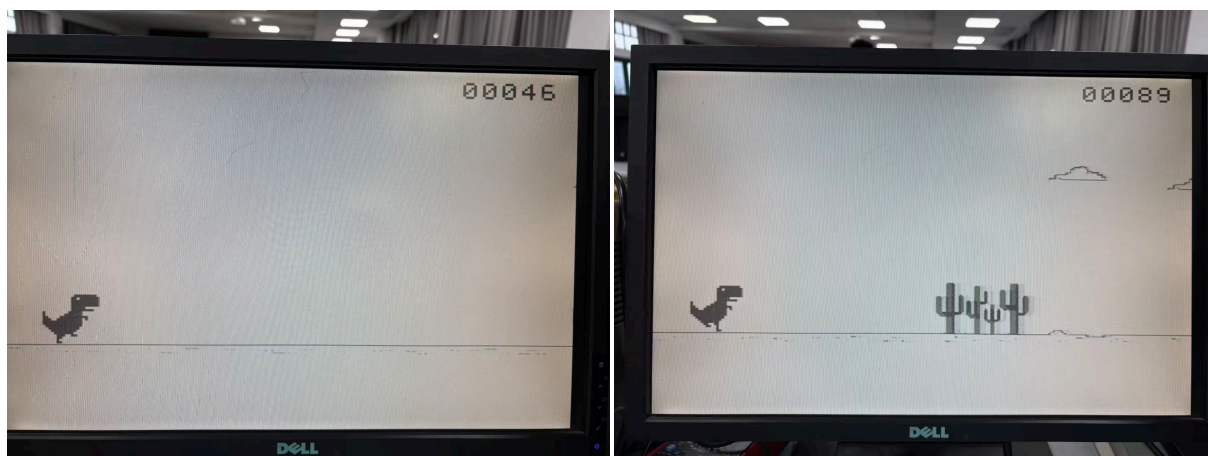


Figure 11 – 画面基本验证结果

上图中，地面能正确地向左连续移动，恐龙能正确切换左右脚交替姿势，仙人掌、翼龙、云朵能随机生成并按相应速度向左移动，右上角的计分板能正确显示得分。

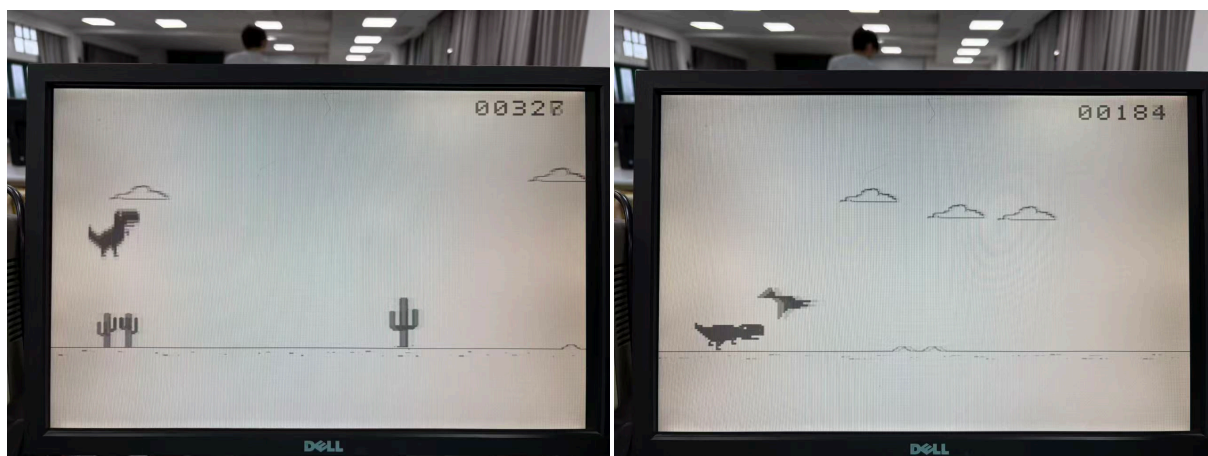


Figure 12 – 恐龙状态验证结果

上图中，恐龙能在按下空格键或上键后进入跳跃状态，按住下键后进入下蹲状态。

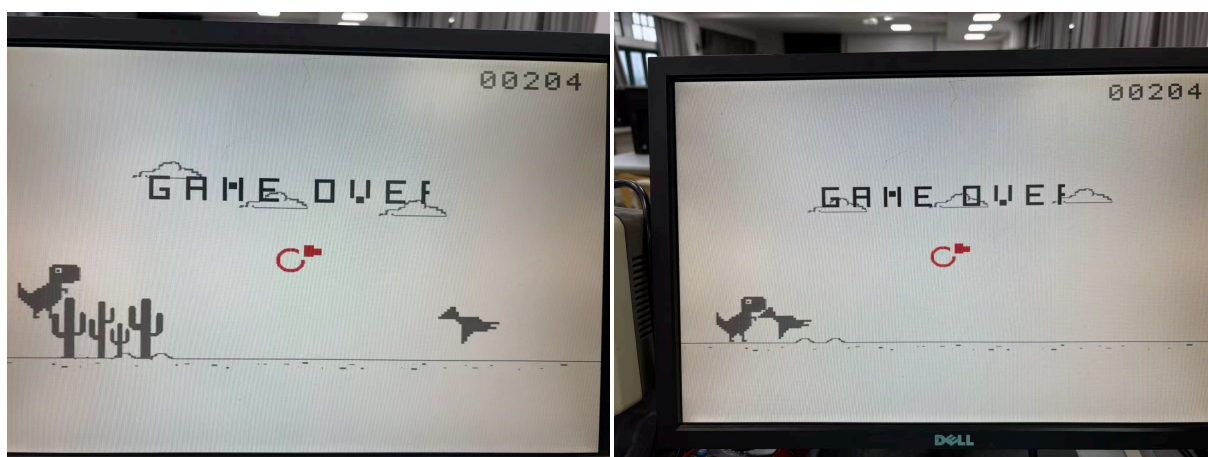


Figure 13 – 碰撞检测验证结果

上图中，当恐龙碰撞仙人掌或翼龙时，游戏能正确进入 Game Over 状态。

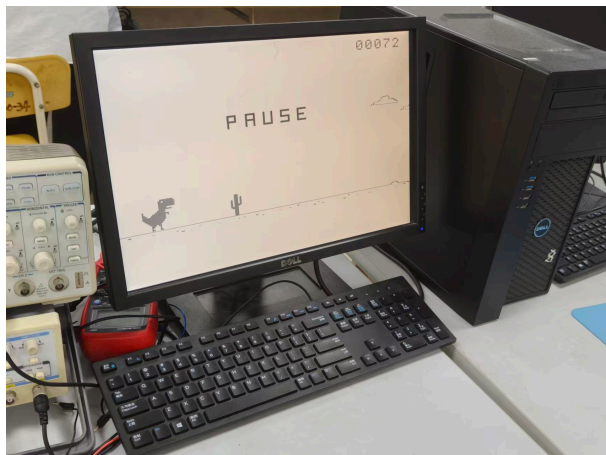


Figure 14 – 暂停功能验证结果

暂停功能是完成项目主体内容后添加的功能。上图中，在“游戏中”状态下按下 P 键，游戏能正确进入“暂停”状态。

VI. 开发与调试过程分析

VI.1. 坐标与素材尺寸

项目初期首先确定屏幕坐标和素材缩放比例。由于 VGA 分辨率为 640×480 ，而原版 Chrome 小恐龙游戏画面较为扁平，如果直接铺满整个屏幕会显得比例不协调。因此本项目将游戏主要区域放在屏幕中下部，地面固定在 $Y=360$ 附近，上方留出云朵、月亮和分数显示区域。

所有主要元素采用原版较大素材的 0.75 倍尺寸。恐龙显示尺寸为 66×70 ，地面贴图宽度为 900，翼龙显示尺寸为 69×60 ，小仙人掌和大仙人掌按类型分别设置宽高。这样既能保持原版比例，又能在 640×480 屏幕上留出足够的反应空间。

VI.2. PS/2 输入

PS/2 的主要问题在于按键通码、断码和扩展码的处理。方向键属于扩展键，需要先接收 E0，再接收具体扫描码；释放按键时还会出现 F0 断码。如果简单地把每个扫描码都作为独立按键处理，会无法区分按下和释放，也不能正确处理方向键。

因此模块中加入 `got_ext` 和 `got_break` 两个标志位，分别记录是否已经接收到扩展码和断码。只有接收到真正的按键扫描码后，才根据这两个标志决定是置位还是清零按键状态。同时，只有在按键原本未按下时才产生触发脉冲，避免长按重复触发。

VI.3. 状态机

状态机调试中曾出现一个问题：当游戏已经处于 `S_PLAY` 状态时，如果此时按下空格，之后发生碰撞，状态机进入 `S_OVER` 后会在下一个 `frame_end` 又立即回到 `S_PLAY`。分析后发现，原因是空格键产生的开始请求被错误保留到了游戏结束状态。

解决方法是将开始请求只限定在 `S_IDLE` 和 `S_OVER` 状态下锁存，并在 `S_PLAY` 或 `S_PAUS` 状态下清除。这样，游戏进行过程中的空格只用于跳跃，不会被解释为下一次游戏开始请求。修改后，仿真和上板验证均表明状态机行为符合预期。

VI.4. VGA 显示

VGA 调试主要围绕时序和像素合成展开。首先通过最小 VGA 工程验证显示器能够识别 $640 \times 480 @ 60\text{Hz}$ 信号，确认 VGA 物理连接和基本时序没有问题。随后再接入游戏显示系统，逐步检查背景、地面、恐龙、障碍物和分数显示。

在像素合成中，如果优先级处理不当，背景可能覆盖前景对象，或者透明像素被当作普通像素显示。为此，所有精灵 ROM 都约定 `00` 表示透明，在 `compose_pixel` 中只对非透明像素进行覆盖，并把恐龙、分数、Game Over 界面放在较高优先级。这样可以保证障碍物、恐龙和界面提示不会被背景元素覆盖。

VI.5. 生成请求跨时钟周期

随机系统产生的 `cactus_new_raw`、`ptero_new_raw` 和 `cloud_new_raw` 是短脉冲，如果运动系统只在 `frame_end` 更新时采样，可能错过这些生成请求。为解决这个问题，游戏逻辑层加入了 `pending` 寄存器，把随机生成请求锁存到下一次帧更新时再交给运动系统消费。这样既保持了随机系统和运动系统之间的解耦，也保证不会丢失生成事件。

VII. 程序流程与操作说明

VII.1. 程序流程

系统复位后，游戏状态为待机，恐龙处于静止姿态，地面坐标回到初始位置，分数清零，障碍物和云朵槽位全部无效。VGA 控制器持续输出画面，显示系统根据当前状态显示待机画面。

当 PS/2 键盘接收到空格或上方向键按下事件时，键盘模块输出单周期触发脉冲。状态机将开始请求锁存，并在下一次 `frame_end` 到来时进入游戏进行状态，同时输出 `game_start` 脉冲。各运动模块收到 `game_start` 后重新初始化坐标和槽位。

游戏进行时，每一帧结束后，系统按以下顺序更新：

1. 状态机根据按键、暂停请求和碰撞信号判断是否切换状态；
2. 恐龙运动模块根据跳跃请求、重力、下蹲键和落地条件更新纵坐标；
3. 随机系统根据 LFSR、当前速度、最小间距和槽位状态决定是否生成新障碍或云朵；
4. 物体运动系统移动地面、仙人掌、翼龙和云朵，并清除移出屏幕的槽位；
5. 碰撞检测模块判断恐龙与有效障碍物是否发生重叠；
6. 计分系统累加分数，并根据分数阶段更新速度；
7. 音效系统根据跳跃、碰撞和得分事件产生蜂鸣器方波。

如果检测到碰撞，状态机在下一帧进入游戏结束状态。此时物体运动停止，显示系统将恐龙切换为死亡姿态，并叠加 `Game Over` 字样和 `Restart` 图标。游戏结束后再次按下空格或上方向键，系统重新进入游戏进行状态。

VII.2. 游戏操作说明

1. 将工程综合、实现并生成 bitstream 或直接使用仓库中的 `top.bit`；
2. 连接 VGA 显示器和 PS/2 键盘，将 bitstream 下载到 Sword 实验板；
3. 上电 / 复位后进入待机状态，按空格键或上方向键开始游戏；
4. 游戏进行中，按空格键或上方向键控制恐龙跳跃；按下方向键控制恐龙下蹲；如果恐龙在空中，按下方向键会加速下落
5. 按 P 键可进入暂停状态或重新开始游戏；
6. 碰撞后进入游戏结束状态，屏幕显示 `Game Over` 字样和重开图标，再次按空格键或上方向键重新开始；

VIII. 总结

本项目总体上采用自底向上为主、自顶向下为辅，自顶向下划分、自底向上验证的设计思路，完成了一个基于 FPGA 的 Google Chrome Dino Game。我们使用 Verilog 实现了 PS/2 键盘输入、VGA 图像输出、主状态机、随机障碍生成、恐龙跳跃、障碍物运动、碰撞检测、计分和蜂鸣器音效等功能。

在实现过程中，我们体会到硬件设计与软件编程的重要区别。很多“瞬间发生”的事件，在硬件中都需要考虑时钟周期和寄存器更新。例如，键盘触发信号只有一个系统时钟周期，而游戏状态只在帧边界更新，如果不进行请求锁存，就可能漏掉按键；反过来，

如果锁存清除条件设计不当，又可能造成游戏结束后立即重开。类似地，随机生成请求、VGA 扫描坐标和游戏物体坐标之间也需要严格区分组合逻辑和时序逻辑。

通过本次课程设计，我们进一步熟悉了 Verilog 模块化设计、有限状态机、ROM 资源读取、VGA 时序、PS/2 协议、仿真 testbench 编写和上板验证流程。最终系统能够实现完整的游戏交互，并具有较直观的 VGA 显示界面和音效反馈，达到了课程设计预期目标。

□ the end